

COMP3211

02. Software Processes

Dr. Max Yu Pei

Intended Learning Outcomes

- ❖ Upon completion of this lecture, you will
 1. understand the concept and importance of software process;
 2. know about the fundamental process activities of software requirements engineering, software development, verification and validation, and evolution;
 3. have been introduced to three general software process models and when they might be used;
 4. understand why processes should be organized to cope with changes in the software requirements and design;
 5. understand the notion of software process improvement and the factors that affect software process quality.

Software Process

- ❖ A structured set of activities required to develop a software system.
- ❖ It is an indispensable pillar of software engineering

You cannot test quality into a product; you must build it in.

- To turn individual heroics into organizational capability
- To create a baseline for auditing and compliance
- To manage the complexity of multiple moving parts over time

Software Process Descriptions

- ❖ When we describe and discuss processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc., and the ordering of these activities.
- ❖ Process descriptions may also include:
 - Products, or deliverables, which are the outcomes of a process activity;
 - Roles, which reflect the responsibilities of the people involved in the process;
 - Pre- and post-conditions, which are statements that are true before and after a process activity has been enacted or a product produced.

Plan-Driven and Agile Processes

- ❖ Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan.
- ❖ In agile processes, planning is incremental, and changing the process to reflect changing customer requirements is easier.
 - In practice, most practical processes include elements of both plan-driven and agile approaches.
 - There are no right or wrong software processes.

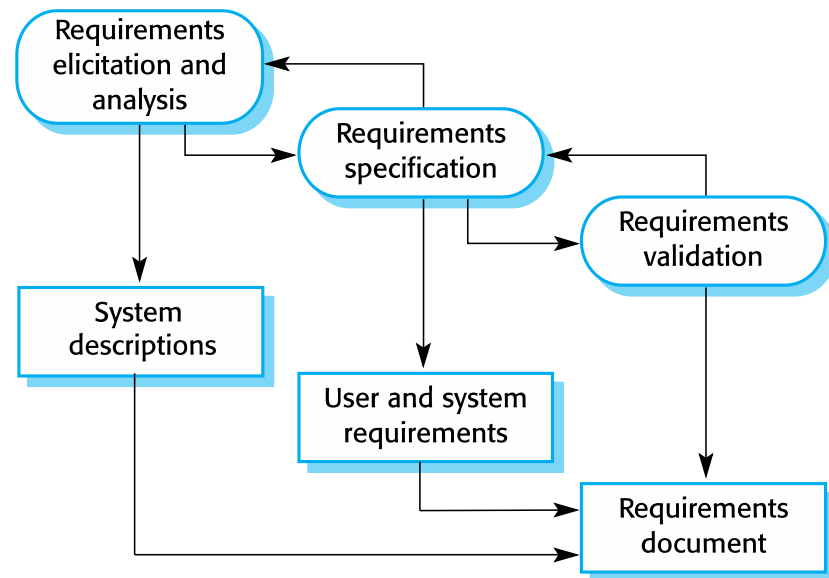
PROCESS ACTIVITIES

Process Activities

- ❖ Real software processes are inter-leaved sequences of technical, collaborative and managerial activities with the overall goal of constructing a software system.
- ❖ There are many different software processes, but they all involve:
 - Specification – defining what the system should do;
 - Development – producing the software that meets the specification;
 - Verification and validation – checking that it conforms to its specification and meets the requirements;
 - Evolution – revising the system to fix errors and to adapt to evolving business or technical demands.

Software Specification

- ❖ The process of establishing what *services* are required and identifying the *constraints* on the system's operation and development.
- ❖ Requirements engineering process
 - Requirements elicitation and analysis
 - What do the system stakeholders require or expect from the system?
 - Requirements specification
 - Defining the requirements in detail
 - Requirements validation
 - Checking the validity of the requirements



The requirements engineering process

Software Development

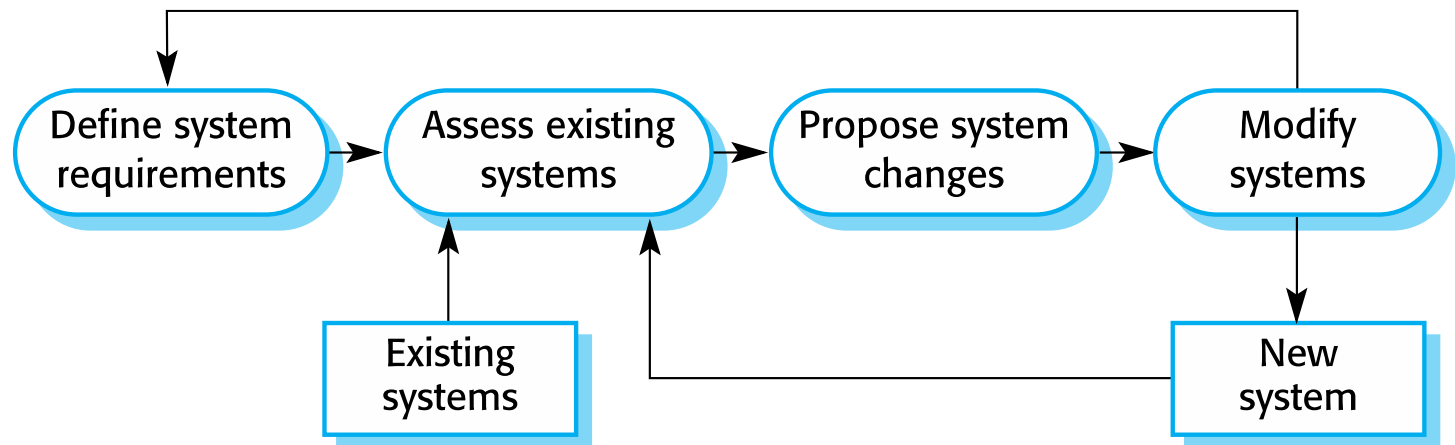
- ❖ The process of developing an executable system for delivery to the customer.
 - Design and implementation are interleaved activities for most types of software systems.
 - Software design
 - Design a software structure to realize the requirements
 - Architectural design, database design, interface design, component selection and design, ...
 - Implementation
 - Translate this structure into an executable program
 - Programming is an individual activity with no standard process

Software Verification and Validation

- ❖ Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer.
 - Verification:
 - Validation:
 - Testing is the most commonly used V & V technique.

Software Evolution

- ❖ Software is inherently flexible and can change.
 - As requirements change, the software that supports the business must also evolve and change.
 - The distinction between development and evolution is increasingly irrelevant as fewer systems are completely new.
 - It is more realistic to think of software engineering as an evolutionary process where software is continually changed in response to changing requirements and customer needs.



SOFTWARE PROCESS MODELS

The Waterfall Model

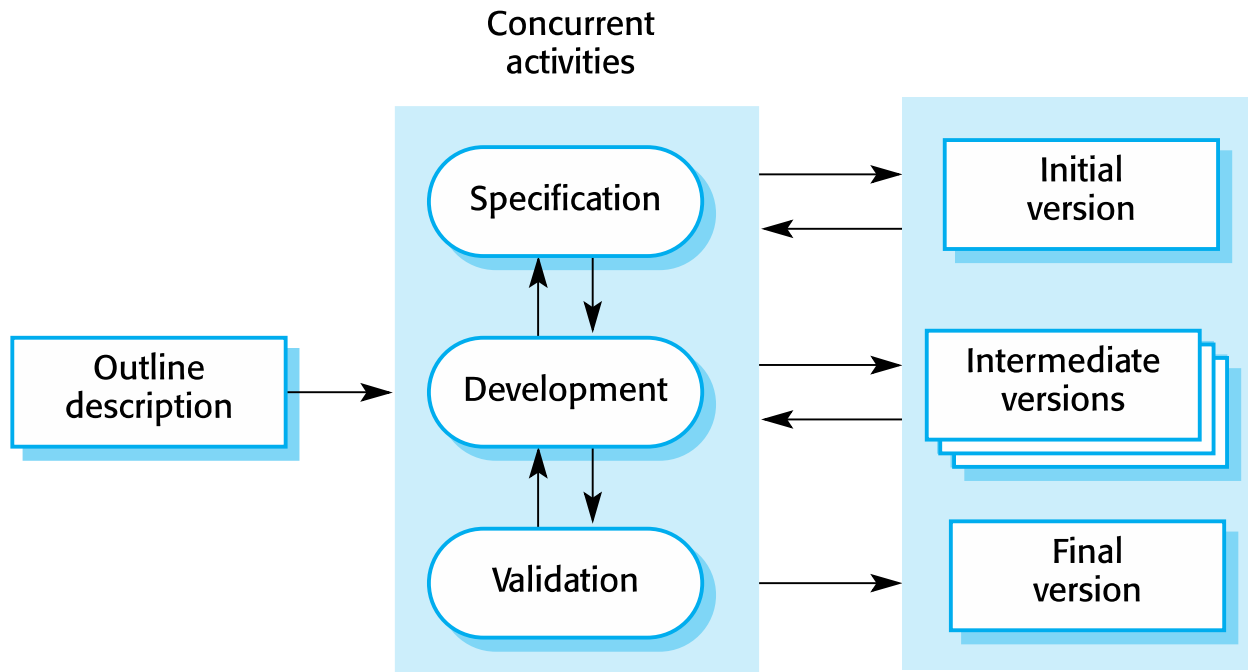
- ❖ There are separate identified phases in the waterfall model, and in principle, a phase must be completed before moving on to the next phase
 - Requirements analysis and definition
 - System and software design
 - Implementation and unit testing
 - Integration and system testing
 - Operation and maintenance

Waterfall Model Problems

- ❖ Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
 - Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
 - Few business systems have stable requirements.
- ❖ Types of systems for which the waterfall model is appropriate
 - Embedded systems where the software has to interface with hardware systems
 - Critical systems where there is a need for extensive safety and security analysis of the software specification and design
 - Large software systems that are part of broader engineering systems developed by several partner companies

Incremental Development

- ❖ Incremental development is based on the idea of developing an initial implementation, getting feedback from users and others, and evolving the software through several versions until the required system has been developed
 - This approach can be either plan-driven, agile or, more usually, a mixture of these approaches.
 - You don't have to deliver each increment to the system customer.



Incremental Development Benefits and Problems

❖ Benefits

- It is easier to get customer feedback on the development work that has been done.
- The cost of accommodating changing customer requirements is reduced.
- More rapid delivery and deployment of useful software to the customer is possible.

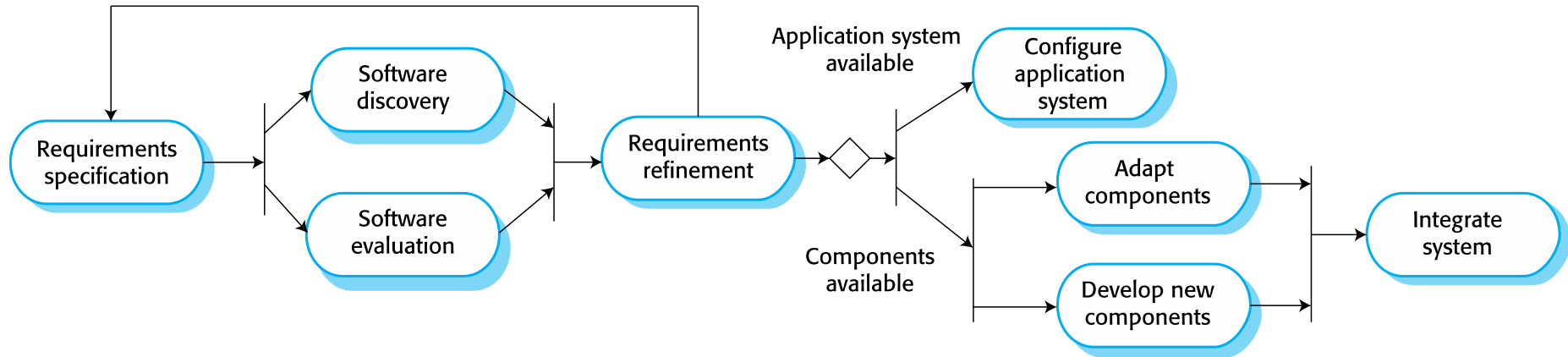
❖ Problems

- It demands stronger configuration management and project coordination.
- The system's initial architecture must be flexible and open to accept new components without breaking existing functionality.
- System structure tends to degrade as new increments are added.

Integration and Configuration

- ❖ Based on software reuse where systems are integrated from existing components or application systems, or COTS (Commercial-off-the-shelf) systems.
 - Reused elements may be configured to adapt their behaviour and functionality to a user's requirements
 - Reuse is now the standard approach for building many types of business systems
- ❖ Types of reusable software
 - Stand-alone application systems that are configured for use in a particular environment.
 - Collections of objects that are developed as a component or a package to be integrated with frameworks like .NET or J2EE.
 - Web services that are developed according to service standards and that are available for remote invocation.

Reuse-Oriented Software Engineering



❖ Advantages and disadvantages

- Reduced costs and risks as less software is developed from scratch
- Faster delivery and deployment of systems
- Requirements compromises are inevitable so the system may not meet the real needs of users
- Loss of control over the system evolution

COPING WITH CHANGE

Coping with Change

- ❖ Change is inevitable in all large software projects.
 - The moving target problem: The requirements change while the software product is being developed
 - There is no solution to the moving target problem
 - The costs of change include both rework and the costs of implementing new functionality
- ❖ Two related approaches to reducing the costs of rework
 - Change anticipation
 - Change tolerance
 - The longer a change is delayed, the more it costs to implement.

Two Ways of Coping with Changing Requirements

❖ System prototyping

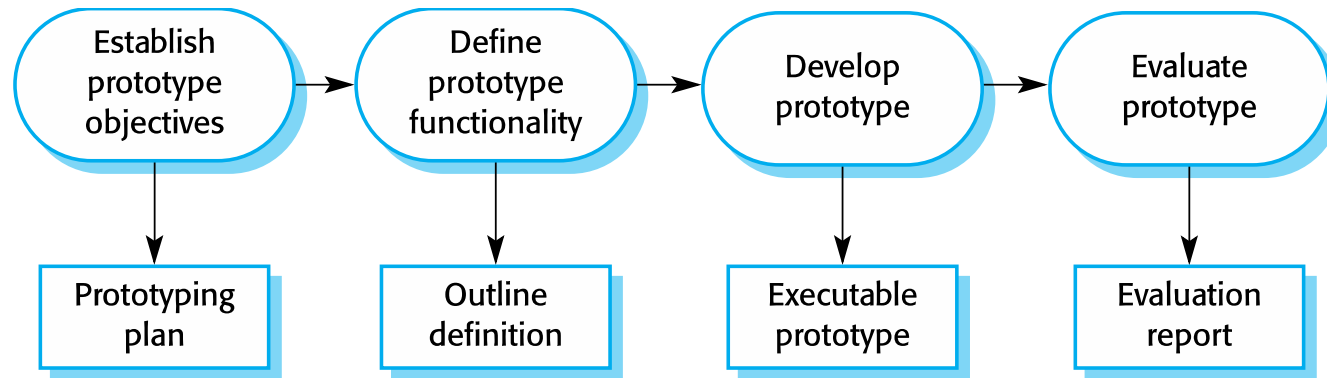
- A version or part of the system is developed quickly to check the customer's requirements and the feasibility of design decisions.
- This is a method of change anticipation as it allows users to experiment with the system before delivery and refine their requirements.

❖ Incremental delivery

- System increments are delivered to the customer for comment and experimentation.
- This supports both change anticipation and change tolerance.

Software Prototyping

- ❖ A prototype is an initial version of a system used to demonstrate concepts, try out design options, and find out more about the problem and its possible solutions.



- ❖ The two strategic approaches
 - Throwaway prototyping (rapid prototyping)
 - Evolutionary prototyping (breadboarding)
- ❖ The golden rule: Prototype just enough to learn, but not enough to ship.

Software Prototyping Advantages and Problems

❖ Advantages

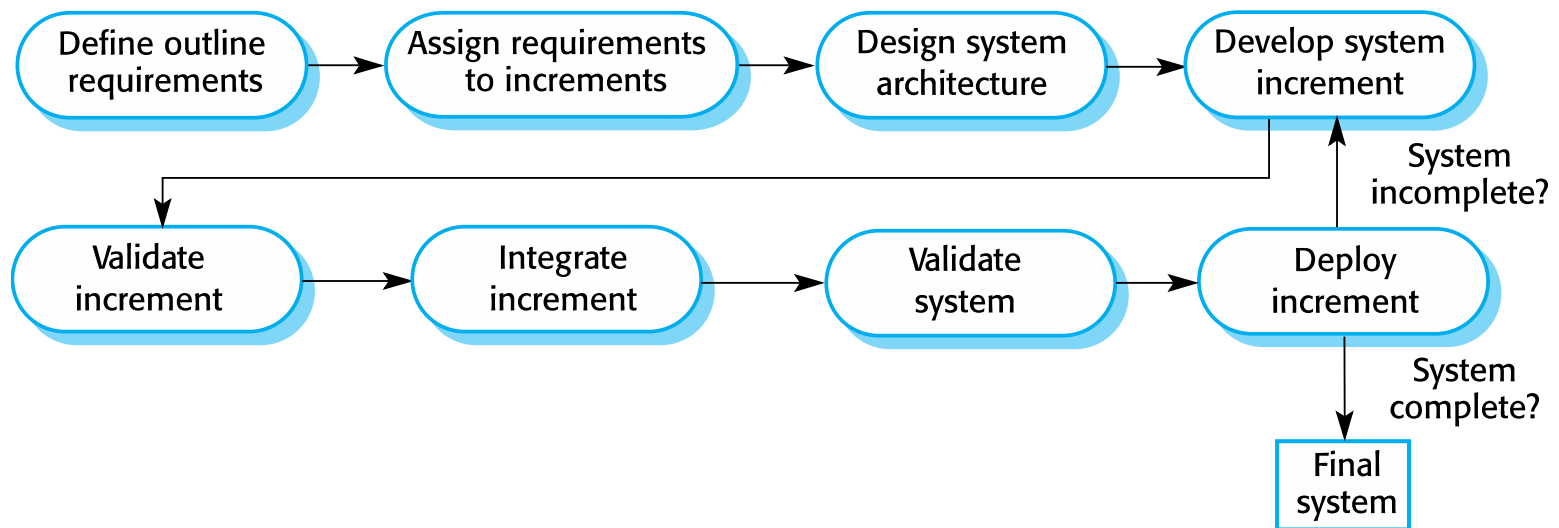
- Clarifies vague requirements
- Early user feedback
- Increases stakeholder buy-in
- Lowers rework costs
- Reduces project risk

❖ Problems

- Stakeholder misinterpretation
- Scope creep
- Increased upfront time & cost
- Analysis paralysis
- False sense of security
- Evolutionary code rot

Incremental Delivery

- ❖ The development and delivery is broken down into increments with each increment delivering part of the required functionality.
 - User requirements are prioritised, and the highest priority requirements are included in early increments.
 - Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve.



Incremental Delivery Advantages and Problems

❖ Advantages

- Early and continuous value delivery
- Faster ROI and cash flow
- Reduced project risk
- Superior feedback loops
- Lower integration shock
- Improved developer morale

❖ Problems

- Architectural erosion (the #1 risk)
- Heavy management & CI/CD overhead
- Scope creep
- Partial utility frustration
- Documentation nightmare
- Fixed-contract nightmares

SOFTWARE PROCESS IMPROVEMENT

Software Process Assessment Dimensions

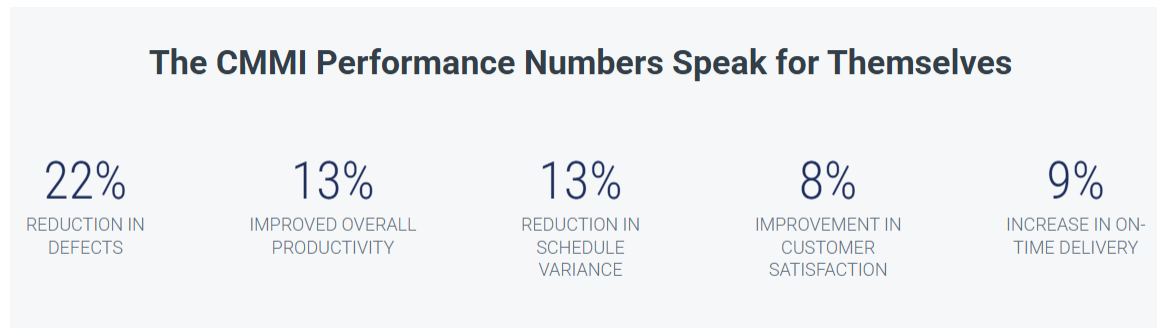
Concept	Level	Definition	Key Question
Performance	Project	The actual, observable results of a specific project or activity. Measured via KPIs like cycle time, defect density, cost, and customer satisfaction.	"What did we actually achieve right now?"
Capability	Specific Process	The inherent, predictable range of results a specific process can produce over time. It answers the "spread" of performance (e.g., we can deliver within $\pm 10\%$ of estimate).	"If we run this process again, what will likely happen?"
Maturity	Organizational	The degree to which an organization has institutionalized, documented, and optimized its processes across the entire enterprise. It is the foundation.	"How robust is our organizational process DNA?"

An Analogy: The Highway Traffic System

- ❖ Performance (the daily commute)
 - The actual time it takes you to get to work this morning.
 - Highly variable and subject to immediate conditions.
- ❖ Capability (the traffic flow predictability)
 - Given the current city condition, can the system reliably get a car from Point A to Point B in a consistent window under normal circumstances?
 - The statistical range, e.g., "95% of trips on this route take between 30 and 40 minutes."
- ❖ Maturity (the city planning department)
 - Does the city have standard policies for road maintenance, synchronized traffic signals, and a central command center?
 - The institutionalized infrastructure that ensures the traffic system is consistent, repeatable, and continuously improved.

CMMI V2.0

- ❖ Capability Maturity Model Integration (CMMI) V2.0 is a widely accepted framework to improve organizational capability and performance.
 - Created by the Software Engineering Institute at CMU, U.S. DoD, and U.S. Government; Currently managed by CMMI Institute

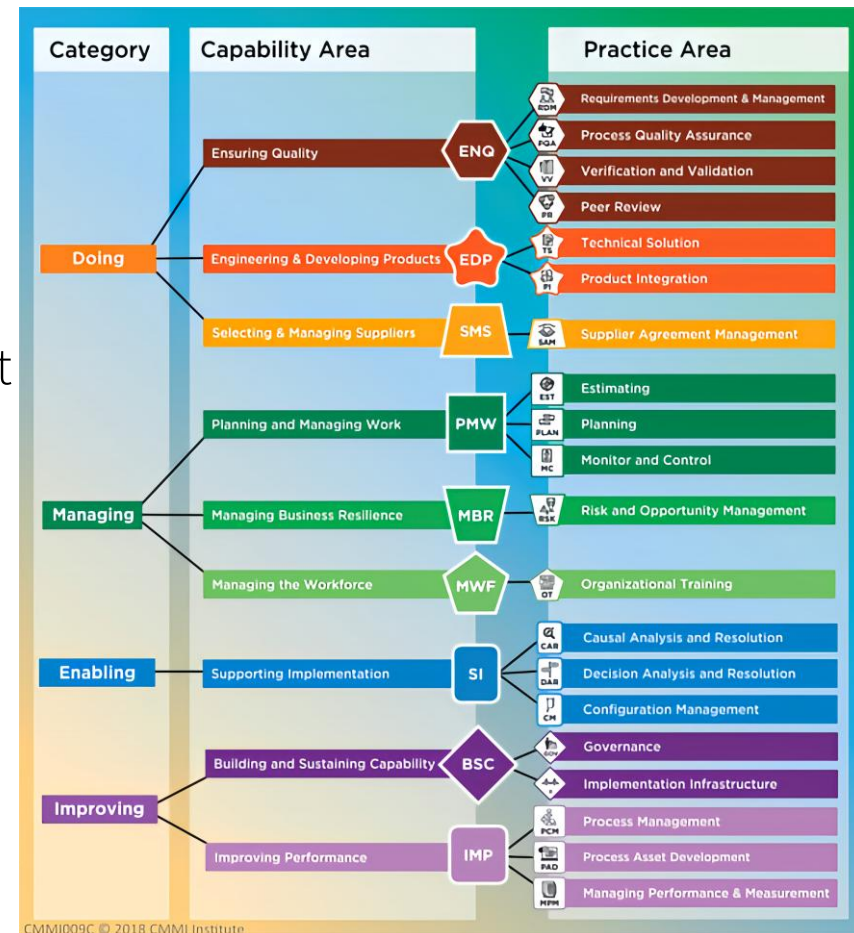


From www.isaca.org

- The goal is to improve performance (better products, faster delivery, happier customers), not just to follow rules.

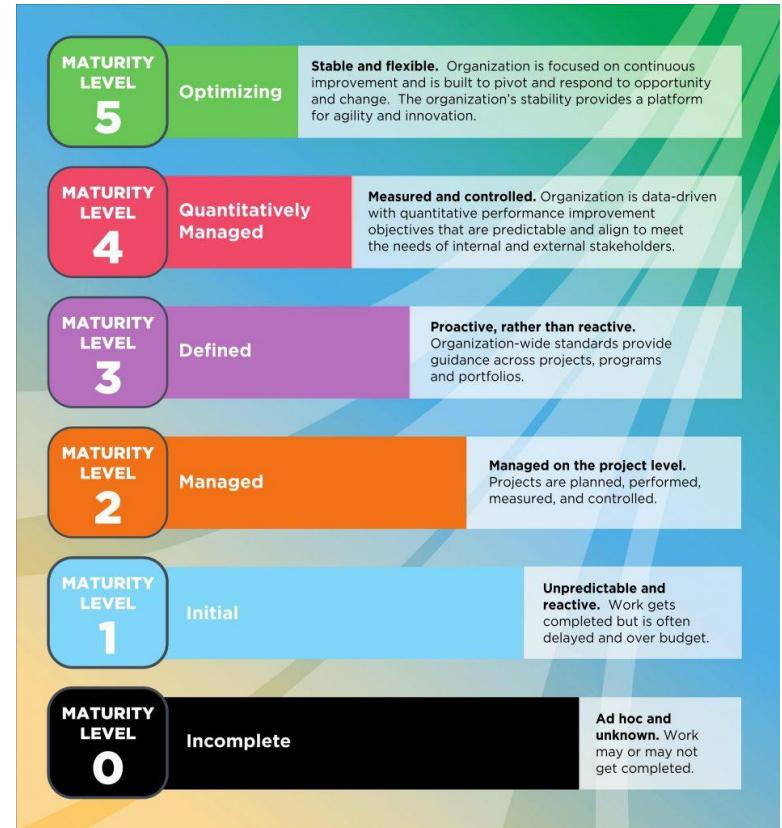
Category, Capability, and Practice Areas

- ❖ CMMI requirements are grouped into categories, capability areas, and practice areas.
 - Each practice area is a set of practices (activities) required to perform a specific discipline
 - Each can be performed at 6 different capability levels (0-5)
 - Capability areas are groups of related PAs that address specific business problems or improvement themes, allowing organizations to adopt the model incrementally.
 - Categories help executives and process owners quickly map improvement efforts to overall business goals.



CMMI Maturity Models

- ❖ CMMI defines 5 CMMI maturity levels.
 - Each maturity level specifies a set of practice areas and the capability levels of their performance
- ❖ Example
 - At maturity level 2, practices from 10 areas, including e.g., Requirements Development & Management (RDM), Planning (PLAN), Monitor and Control (MC), must be performed at capability levels 1 or 2
 - Higher maturity levels require more practice areas to be performed and at higher capability levels



The Application of CMMI V2.0

- ❖ The _____ improvement approach
 - Set a strategic goal to achieve a specific maturity level
 - Systematically improve all the practice areas required

- ❖ The _____ improvement approach
 - Benchmark & diagnose
 - Prioritize & set goal
 - Implement & integrate
 - Measure & improve

The Improvement Progression

Maturity Level	Organizational State	Resulting Process Capability	Resulting Project Performance
Level 1: Initial (Ad-hoc)	Processes are reactive, undocumented, and depend entirely on individual heroics.	Unpredictable. High variance; capability is essentially undefined (0% predictability).	Chaotic / Risky. Performance is a coin-toss. Success is often accidental.
Level 2: Managed (Project-level)	Processes are planned, tracked, and controlled at the project level. Basic project management (requirements, planning, monitoring) is in place.	Somewhat stable. Past performance on this specific project can be repeated for similar work.	Repeatable. Performance is consistent within the same team/project, but varies across different projects.
Level 3: Defined (Organizational)	Processes are standardized across the entire organization. There is a common training curriculum and standard process framework.	Standardized. Variance narrows significantly across all teams because everyone uses the same playbook.	Consistent. Performance is reliable, regardless of which project team executes the work.
Level 4: Quantitatively Managed (Data-driven)	The organization uses statistical and quantitative techniques (e.g., Six Sigma) to control sub-processes.	Measurably stable. Process capability limits (e.g., control charts) are formally established and tracked in real-time.	Controlled. Performance stays within strict quantitative bounds (e.g., 99.7% within tolerance).
Level 5: Optimizing (Continuous Improvement)	The organization uses the quantitative data from Level 4 to proactively tweak, adjust, and innovate the standard processes.	Continuously improving. The statistical range of capability actively tightens year-over-year (less waste, fewer defects).	World-Class. Performance not only improves, but the extremes (worst-case scenarios) are systematically eliminated.

Quality Assurance (QA)

❖ Quality Assurance

- the identification, planning, adoption, and implementation of the disciplines and actions (the process) necessary to assure that the product satisfies its users.
- CMMI 2.0 tells the organization what to do to achieve process maturity;
- QA is the specific mechanism that ensures it actually gets done and provides the objective evidence required for appraisal
 - Use error and trend analysis to locate weaknesses in various processes

Key Points

- ❖ Software processes are the activities involved in producing a software system.
- ❖ Software verification and validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.
- ❖ General process models describe the organization of software processes. Examples of these general models include the 'waterfall' model, incremental development, and reuse-oriented development.
- ❖ Processes should include activities such as prototyping and incremental delivery to cope with change.
- ❖ CMMI V2.0 identifies maturity levels that essentially correspond to the adoption of good software engineering practice.

THE END